10 Aug, 2024

Class 22

AJAX & APIs

↓

Async JS & XML → markup language

↓ data

XML / JSON

↳ object of JS converted to string

(But it doesn't have function)

↓ security reasons

dummy api

get request → fetch data

post request → Server par data change karne ke liye

web url sirf se get request hi hota hai.

fetch, axios, xml http req

jsonplaceholder typicode todo → api

hamne api se data call kiya.

https://jsonplaceholder.typicode.com/todos

query parameter — req → header, body, params

path "

2xx → ok

4xx → client end error

5xx → server / backend end error

→ client error responses → mdn

kisi bhi data (object) ko JSON kaise banaye?

stringify / parse

Stringify hone par function halt jata hai.

get
post → create
put → purana delete kiya, new add kiya
patch → mobile update
delete

axios vs fetch

XHR ⇒ No need for request

Homework - ① Todo Api & data mang kar Apne todo mei data use kar rahe he.

② Api movie
↳ Api based project banao

IIFES :-

↳ we use it to hide data from user.

when do we use iife?

Shreyance Lec 18 | Personal Notes | 13 Aug, 2024

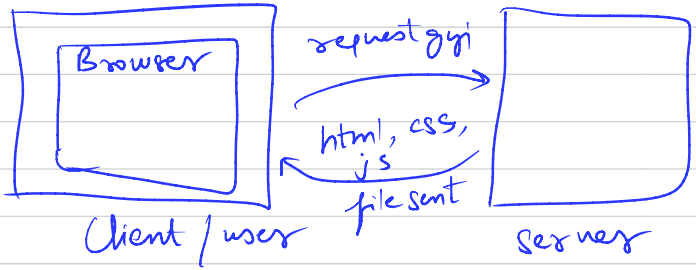
AJAX and API (Timestamp : 2:20:00)

AJAX is a technique jiske through hum request ko send kar sakte hai without disturbing the current webpage.

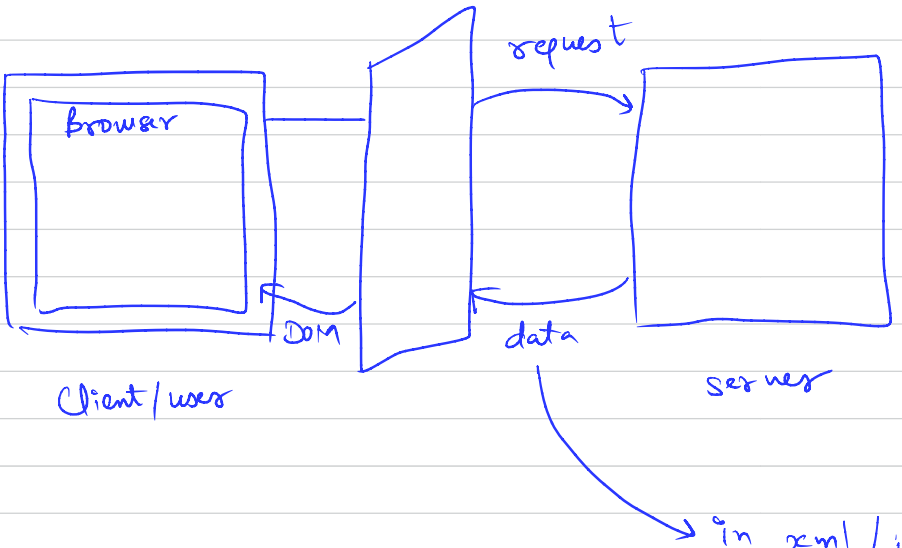
AJAX → Asychronous JavaScript and XML

XML → extensible markup language

Jab mai google par kuch search / ya koi website open karta hu, then website ke server par se html, css, js ki file request ki jati hai. These files are rendered by browser. Browser ke paas JS engine and layout engine hota hai. Layout engine html, css, js ki received files from server ko render karta hai. The files are requested when user searches for something on internet.



AJAX Javascript ki layer provide karta hai b/w user/client and server



xml files pehle use hoti thi, ab json file use hoti hai.

Browser is connected to JS layer.
JS ki layer server ko request bhejegi. Server layer data return karega either in form of xml file or json file. In current days, it will be json file. Now using DOM, we will make changes in data file (xml/json)

API :- (Application Programming Interface)

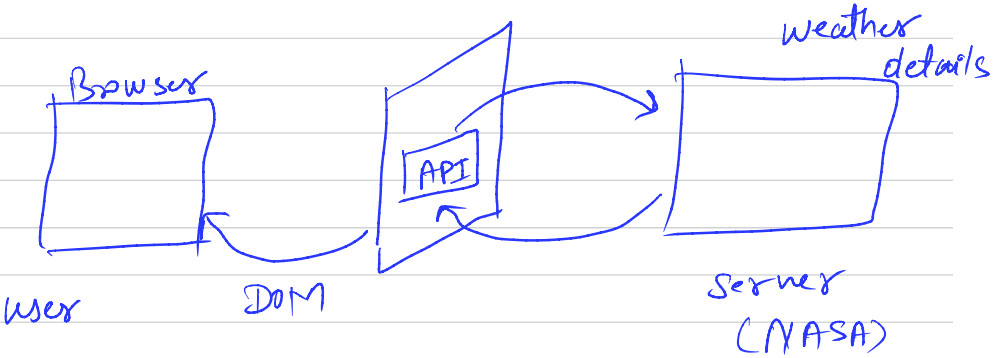
API full form is an Application Programming Interface that is a collection of communication protocols and subroutines used by various programs to communicate between them.

let's take a real-life example of an API, you can think of an API as a waiter in a restaurant who listens to your order request, goes to the chef, takes the food items ordered and gets back to you with the order

→ In this example, you are getting data/food but you dont know ki jo food prepare hua hai vo kaise prepare hua hai. you are only getting the final result, not the process.

API helps in communication b/w two softwares / programs.

Let's say you want to show NASA weather forecast data on your website. Then you will use NASA Api to fetch the data.



API provides you detail in json file format.

you can get dummy api from dummyjson.com

dummyjson.com/products → it is an api

Ab mai API data ko DOM ke through mani pulate kar sakta hu. yaha page reload bhi nahi hoga. Isi ko hum AJAX kehte hai.

AJAX is too old, we dont use it.
we only use fetch Api, axios.

(lecture - 19 Shreyance online)

API continues

get public Apis from github.

Search "github public api" → open first link → scroll down to find animals → open cat-facts → open "start develo ping!"

you will see the API end point

↓

check ss on next page

View on GitHub

API Documentation

Base URL for all endpoints `https://cat-fact.herokuapp.com` → Base URL

The response time will likely be a few seconds long on the first request, because this app is running on a free Heroku dyno. Subsequent requests will behave as normal.

Endpoints

`/facts` Retrieve and query facts

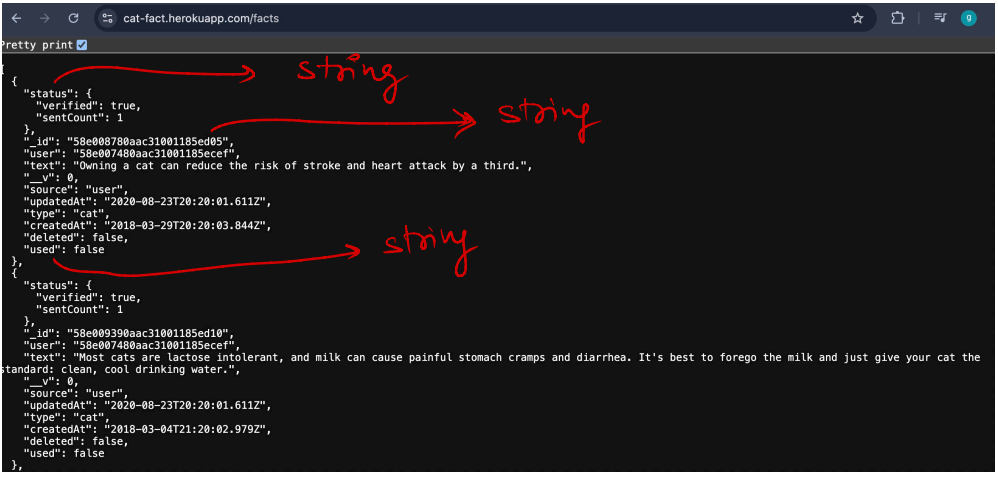
`/users` * Get user data

] → end points

* Requires authentication. As of now, this can only be achieved by logging in manually on the website.

https://cat-fact.herokuapp.com/facts

JSON file is stringified object of Java script.




```
{
  "status": {
    "verified": true,
    "sentCount": 1
  },
  "_id": "58e008780aac31001185ed05",
  "user": "58e007480aac31001185ecef",
  "text": "Owning a cat can reduce the risk of stroke and heart attack by a third.",
  "__v": 0,
  "source": "user",
  "updatedAt": "2020-08-23T20:20:01.611Z",
  "type": "cat",
  "createdAt": "2018-03-29T20:20:03.844Z",
  "deleted": false,
  "used": false
},
{
  "status": {
    "verified": true,
    "sentCount": 1
  },
  "_id": "58e009390aac31001185ed10",
  "user": "58e007480aac31001185ecef",
  "text": "Most cats are lactose intolerant, and milk can cause painful stomach cramps and diarrhea. It's best to forego the milk and just give your cat the standard: clean, cool drinking water.",
  "__v": 0,
  "source": "user",
  "updatedAt": "2020-08-23T20:20:01.611Z",
  "type": "cat",
  "createdAt": "2018-03-04T21:20:02.979Z",
  "deleted": false,
  "used": false
},
```


you can add (install) json formatter extension to make json file neat and well formatted

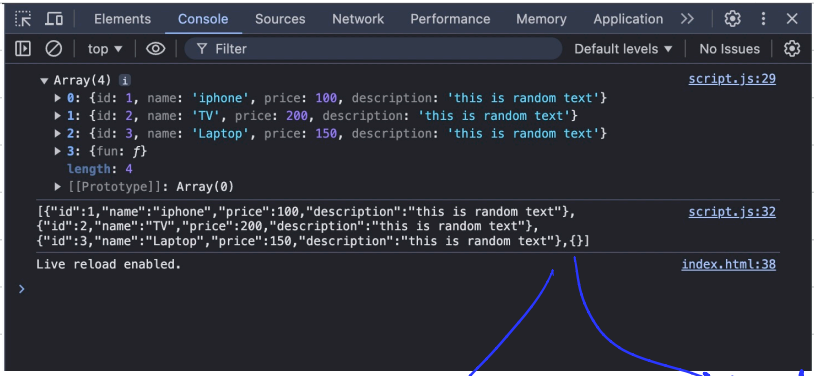
```
const product = [
    {
        id:1,
        name:'iphone',
        price:100,
        description:'this is random text'
    },
    {
        id:2,
        name:'TV',
        price:200,
        description:'this is random text'
    },
    {
        id:3,
        name:'Laptop',
        price:150,
        description:'this is random text'
    },

    {
        fun: function hacked(){
            console.log('You are hacked!!')
        }
    }

];

console.log(product);

const stringProduct = JSON.stringify(product);
console.log(stringProduct);
```

connecting to string

yaha array aayega as it is

yaha bhi array aayega but with stringified object

O/p ↓



empty

no function is there

Stringify karne par function remove ho gya. Security reasons ke karan aisa hota hai.

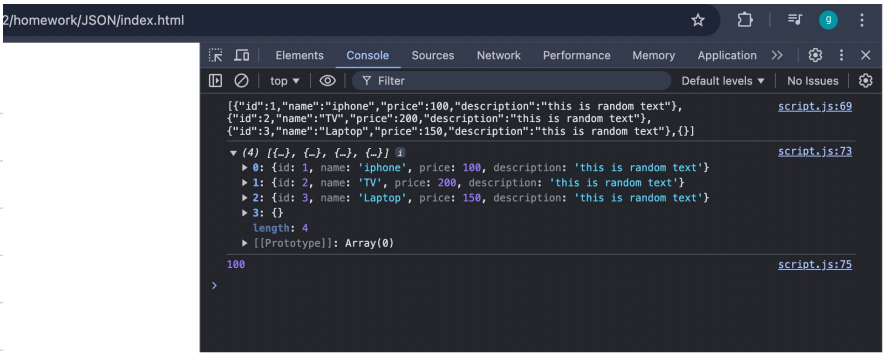
Maan lo aapne api call kari and usme malware wala function hai and agar vo aapke system mei run ho gya, then your pc will be hacked. Isliye functions are removed during stringify.

Stringify ko reverse karne ke liye use parse.

const parseProduct = JSON.parse(stringProduct);

```
const product = [
    {
        id:1,
        name:'iphone',
        price:100,
        description:'this is random text'
    },
    {
        id:2,
        name:'TV',
        price:200,
        description:'this is random text'
    },
    {
        id:3,
        name:'Laptop',
        price:150,
        description:'this is random text'
    },

    {
        fun: function hacked(){
            console.log('You are hacked!!')
        }
    }

];


const stringProduct = JSON.stringify(product);
console.log(stringProduct);


const parseProduct = JSON.parse(stringProduct);
console.log(parseProduct);

console.log(parseProduct[0].price);
```

O/p ⇒



Request :-

3

AJAX request bhejne ke ~~do~~ tarike hai

↳ ① Fetch
↳ ② Axios
→ they both are used

↳ ③ http XML (too old)
↳ outdated

Fetch hamesha promise return karta hai.

Status 200 → means everything is okay.

↳ it is a webapi.

```
const URL = 'https://cat-fact.herokuapp.com/facts';

const btn = document.querySelector('button');

btn.addEventListener('click', (e)=>{
    fetch(URL)
            .then((res)=>{
                return res.json();
            })
            .then((data)=>{
                console.log(data);
                data.forEach(obj => {
                    console.log(obj.text);
                });
            })
            .catch((err)=>{
                reject(err);
            })
})
```

l1 l2 l3 l4 l5 l6 l7

console.log(err)

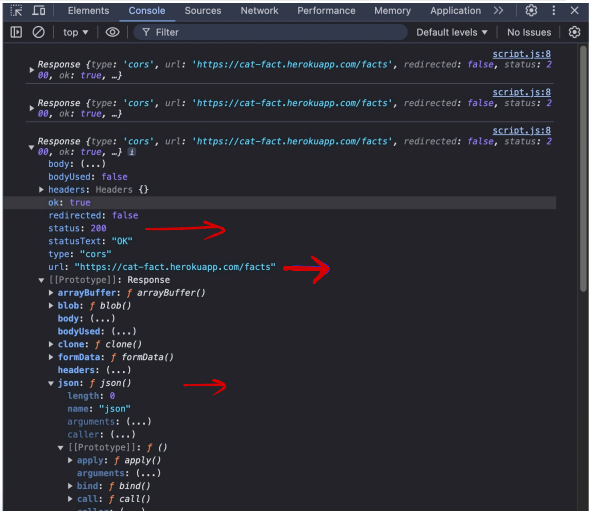
(43:06)

l1 → fetch URL par request bhejega and fetch ek promise return karega. Promise ko handle karne ke liye .then, .catch use hoga.

l2, l3 ⇒ l2 par res → ye ek object hai and iss object ke ander json function hai

Learning Fetch

Click Me



Upar wale screenshot mei hamne Response object console.log karaya.

```
console.log(res)
```

Status → 200 which means API is working fine

Prototype ke ander json function hai.

l3 → l3 par res.json() promise return kar raha hai and again hame .then, .catch use karna padega.

The fetch() function sends a GET request to the URL defined earlier.
fetch() returns a promise that resolves to a response object (res).
console.log(res) logs the raw response to the console.
res.json() is called to parse the JSON body from the response, returning another promise.

The parsed JSON data is passed to the next .then() block as data. l3, l4
console.log(data) logs the entire array of objects to the console. l5
The forEach() method iterates over the array, logging each text property of the objects in the array to the console. l6

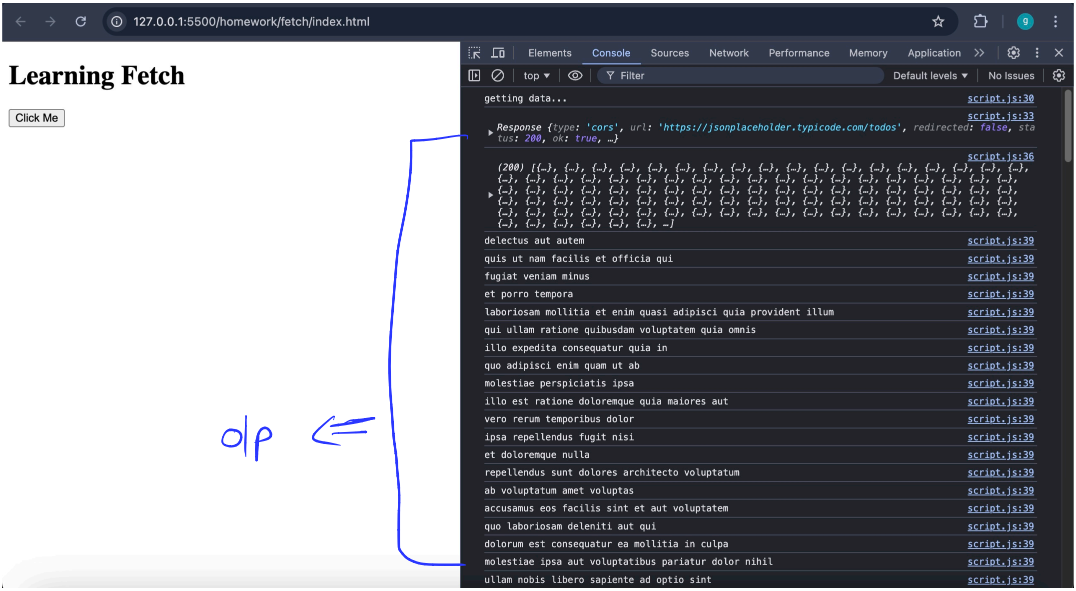
If there is any error during the fetch operation or in processing the data, the .catch() block will handle it. l7
reject(err) is likely intended to pass the error to another promise (but reject is undefined in this scope, so it may cause an error; perhaps it should be console.error(err) instead).

→ The code has been corrected

43:06

Humne Api call karne ke liye pehle promise use kiya tha, ab hum async await use karenge.

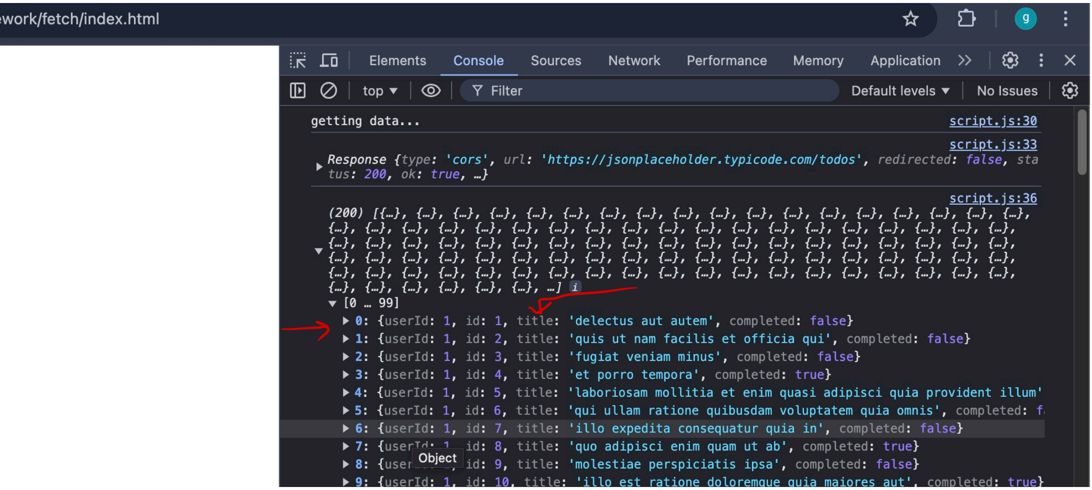
```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const getFacts = async ()=>{
    console.log('getting data...');
    let response = await fetch(URL);

    console.log(response);

    let data = await response.json();
    console.log(data);

    data.forEach(obj => {
        console.log(obj.title);   → l1
    })
}

getFacts();
```



l1 par aap decide karoge ki title aayega ya kuch aur?

Data ko console karane par, you will need to open the data and look for key name which

you want to see as o/p.

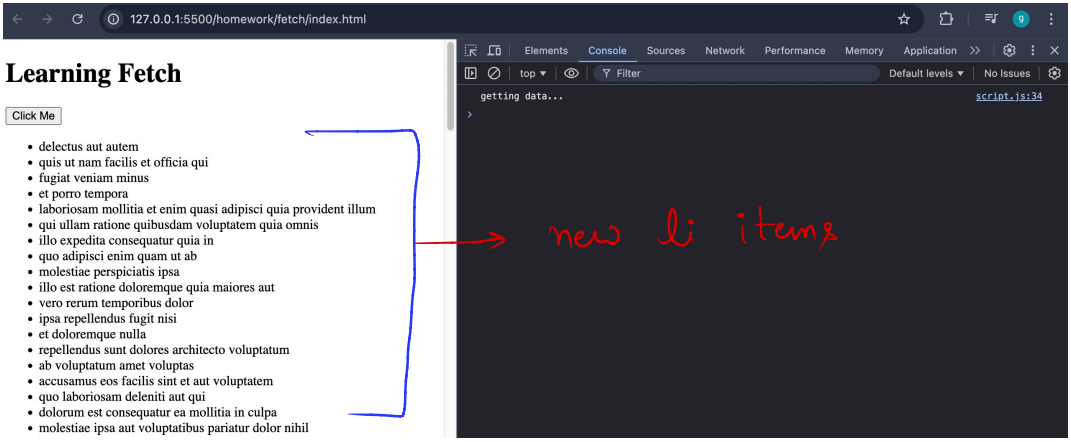


Note:- Cat fact wali Api work nahi kar rahi islye humne to do wali Api use kari.

Q Text mei mile data ko html ke ul mei append karo.

```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const btn = document.querySelector('button');

const catFacts = document.querySelector('.cat-facts');

const getFacts = async ()=>{
    console.log('getting data...');
    let response = await fetch(URL);

    // console.log(response);

    let data = await response.json();
    // console.log(data);

    data.forEach(obj => {
        // console.log(obj.title);
        const newLi = document.createElement('li');
        newLi.innerText= obj.title;
        catFacts.append(newLi);
    })
}

btn.addEventListener('click', getFacts)
```

getFacts function will be called when button is clicked and li items will be appended.




```
const URL = 'https://jsonplaceholder.typicode.com/
todos';

const btn = document.querySelector('button');

const catFacts = document.querySelector('.cat-
facts');

btn.addEventListener('click', (e)=>{
    fetch(URL)
            .then((res)=>{
                // console.log(res);
                return res.json();
            })

            .then((data)=>{
                // console.log(data);
                data.forEach(obj => {
                // console.log(obj.title);
                const newLi =
document.createElement('li');
                newLi.innerText= obj.title;
                catFacts.append(newLi);
            })

            })
            .catch((err)=>{
                console.log(err);
            })
})
```

→ you can use this code. you will get the same o/p. Here we have used promise

Note:- you can also do error handling with async await.

```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const btn = document.querySelector('button');
const catFacts = document.querySelector('.cat-facts');

// Define the async function to fetch data and update the DOM
const getFacts = async () => {
    try {
        console.log('Getting data...');
        let response = await fetch(URL);

        if (!response.ok) {
            throw new Error('Network response was not ok');
        }

        console.log(response);

        let data = await response.json();
        console.log(data);

        // Clear any existing content in the list
        catFacts.innerHTML = '';

        // Populate the list with fetched data
        data.forEach(obj => {
            console.log(obj.title);
            const newLi = document.createElement('li');
            newLi.innerText = obj.title;
            catFacts.append(newLi);
        });
    } catch (error) {
        console.error('There was a problem with the fetch operation:', error);
    }
};

// Attach the event listener to the button
btn.addEventListener('click', getFacts);
```

upar wale code ko aap aise bhi likh sakte ho:-

This is a professional way of writing the code.

```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const btn = document.querySelector('button');
const catFacts = document.querySelector('.cat-facts');


function getData(URL){
    return new Promise((resolve, reject) => {
        fetch(URL)
            .then((res)=>{
                return res.json();
            })
            .then((data)=>{
                resolve(data);
            })
            .catch((err)=>{
                reject(err);
            })
    })
}

function addDataToList(factsData){
    catFacts.innerText = '';
    factsData.forEach( title =>{
        const li = document.createElement('li');
        li.innerText = title;
        catFacts.append(li);
    })
}


function handler(){
    getData(URL)
        .then((data)=>{
            const factsData = data.map((obj)=>{
                return obj.title;
            })

            console.log(factsData);
            addDataToList(factsData);
        })
        .catch(err => console.log(err));
}

btn.addEventListener('click', handler)
```

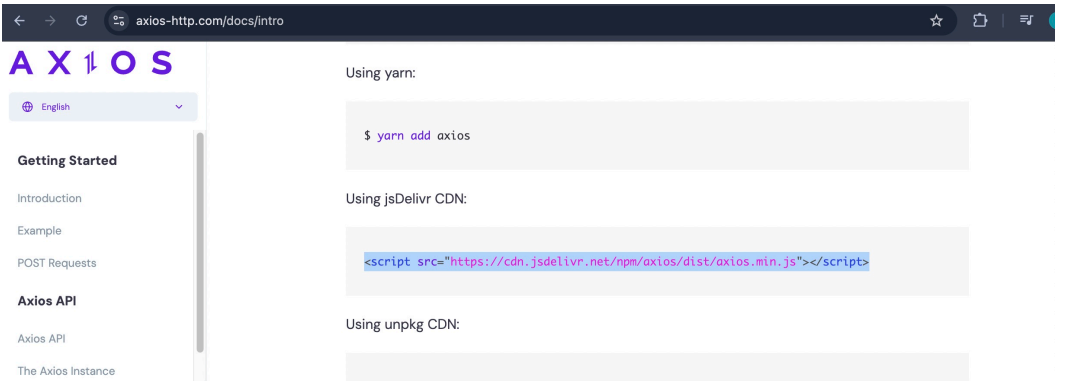
Q write a code using async await for the same thing (done above) in a professional way.

```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const btn = document.querySelector('button');
const catFacts = document.querySelector('.cat-facts');

async function getData(URL){

    try {
        const res = await fetch(URL);
        const data = await res.json();
        return data;

    } catch (error) {
        console.log(error);
    }

}


function addDataToList(factsData){
    catFacts.innerText = '';
    factsData.forEach( title =>{
        const li = document.createElement('li');
        li.innerText = title;
        catFacts.append(li);
    })
}

async function handler(){
    try {
        const data = await getData(URL);
        const factsData = data.map((obj)=>{
            return obj.title;
        });
        console.log(factsData);
        addDataToList(factsData);

    } catch (error) {
        console.log(error);
    }

}


btn.addEventListener('click', handler);
```

1°.40 : 00

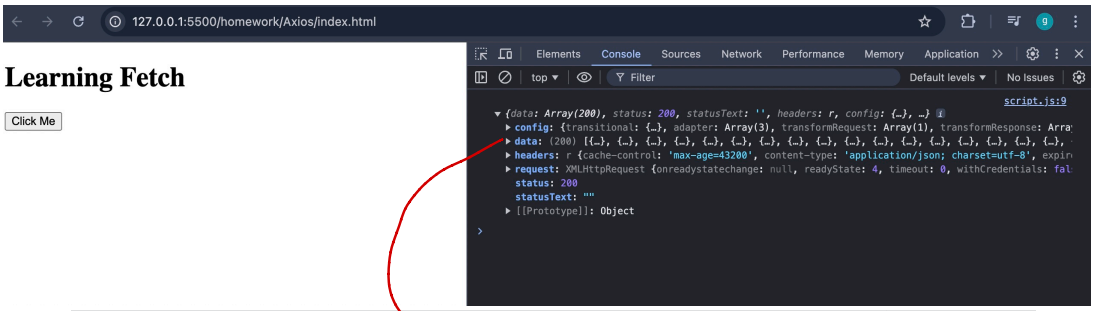
Axios :- ye bhi promise return karta hai.

Axios ka CDN apni html file ke header mei daalo.




```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const btn = document.querySelector('button');
const catFacts = document.querySelector('.cat-facts');

btn.addEventListener('click', (e)=>{
    axios.get(URL)
        .then((res)=>{
            console.log(res);
        })
});
```

o/p

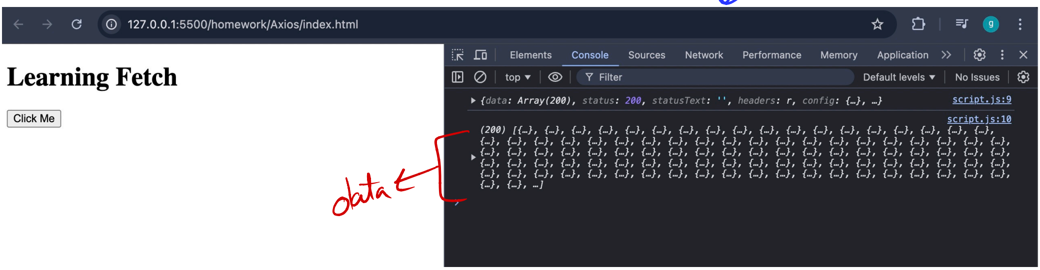


yaha data mil gya.
bahar

Fetch ke case mei json() ko call karna pad raha tha.
Yaha res.data karne par hi data mil jayega.

```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const btn = document.querySelector('button');
const catFacts = document.querySelector('.cat-facts');

btn.addEventListener('click', (e)=>{
    axios.get(URL)
        .then((res)=>{
            console.log(res);
            console.log(res.data);
        })
});
```

o/p



1:46:00

Sir ne fetch ke code ko axios ke code mei convert karaya.

```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const btn = document.querySelector('button');
const catFacts = document.querySelector('.cat-facts');

function getData(URL){
    return new Promise((resolve, reject) => {
        axios.get(URL)
            .then((res)=>{
                resolve(res.data);
            })
            .catch((err)=> reject(err));
    })
}

function addDataToList(factsData){
    catFacts.innerText = '';
    factsData.forEach( title =>{
        const li = document.createElement('li');
        li.innerText = title;
        catFacts.append(li);
    })
}

function handler(){
    getData(URL)
        .then((data)=>{
            const factsData = data.map((obj)=>{
                return obj.title;
            })

            console.log(factsData);
            addDataToList(factsData);
        })
        .catch(err => console.log(err));

}

btn.addEventListener('click', handler)
```

⇒ this part

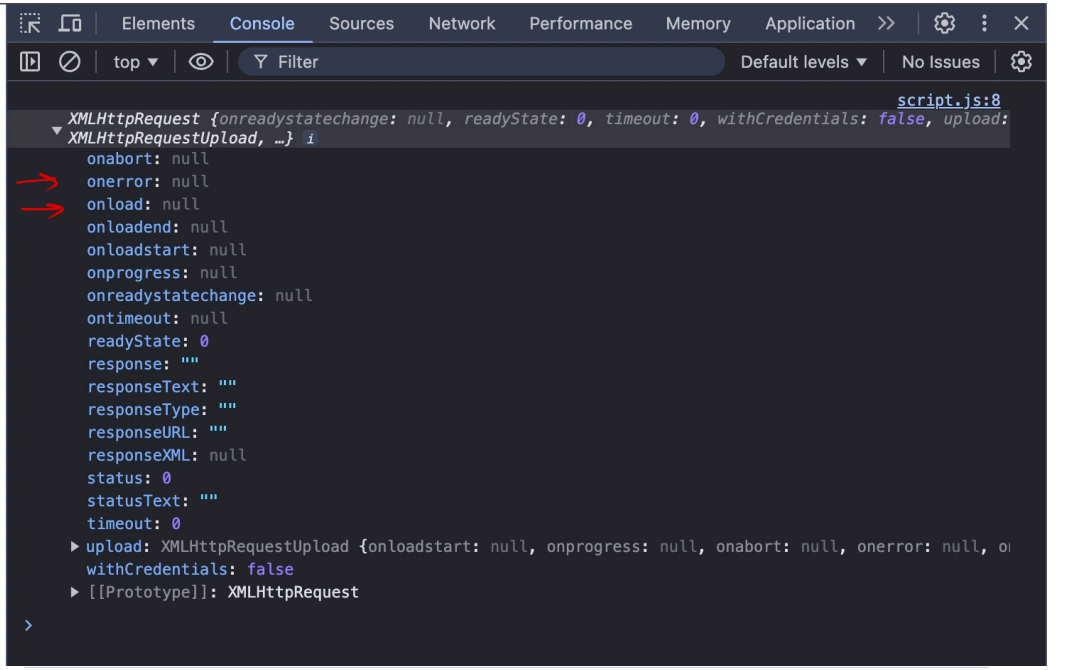
Iss tarike se code likhne ka fayeda kya hai?
Agar axios / fetch ki jagah kuch aur naya aa gya then you would need to change the highlighted part only ("check this part in the code above")

Aapko poora code change nahi karna padega.

XML http Request :-

const xml = new XMLHttpRequest();

a new object xml is created. This object has many keys set as null.



```
const URL = 'https://jsonplaceholder.typicode.com/todos';

const btn = document.querySelector('button');
const catFacts = document.querySelector('.cat-facts');

const xhr = new XMLHttpRequest();

// console.log(xhr);

// request ke success hone par ye chalega
xhr.onload = function(res){
    console.log(res);
    const data = res.currentTarget.response;
    console.log(data);
    console.log(JSON.parse(data));
}

// request ke fail hone pr ye chalege
xhr.onerror = function(err){
    console.log(err);
}

// request kha bhejni hai, define karo
xhr.open('GET', URL);

btn.addEventListener('click', ()=>{
    // request ko send karna
    xhr.send();
})
```

code explanation is on next page :-

Step-by-Step Explanation

Setting the URL: The URL variable is defined with the value 'https://jsonplaceholder.typicode.com/todos'. This URL is an endpoint where the code will send a GET request to fetch some data. The specific URL provided here is a placeholder API that returns a list of to-do items in JSON format.

Selecting the Button and Cat-Facts Container:

The code then selects an HTML element, likely a button, using document.querySelector('button'), and stores it in the btn variable.
Another element is selected with the class .cat-facts, which is stored in the catFacts variable. This element will likely be used later to display data fetched from the API.
Creating an XMLHttpRequest Object: An XMLHttpRequest object is created and stored in the xhr variable. This object is used to interact with servers and allows you to send and receive data asynchronously.

Handling Successful Requests:

The xhr.onload function is defined to handle the case when the HTTP request is successfully completed.
When the request is successful, the function logs the response object (res) to the console.
Then, it retrieves the response data (likely in JSON format) from res.currentTarget.response and logs it.
The raw JSON data is then parsed into a JavaScript object using JSON.parse(data), which is also logged to the console. This parsed object is easier to work with programmatically.
Handling Errors:

The xhr.onerror function is defined to handle any errors that might occur during the request.
If the request fails, the function logs the error (err) to the console. This helps in debugging issues related to network requests.
Configuring the Request:

The xhr.open('GET', URL) method is called to set up the request.
The first argument, 'GET', specifies the type of HTTP request to make (in this case, a GET request).
The second argument, URL, specifies the endpoint where the request will be sent.
Sending the Request on Button Click:

An event listener is added to the button (btn) that listens for a 'click' event.
When the button is clicked, the function inside the event listener is triggered.
Inside this function, the xhr.send() method is called to send the request to the server.
Summary

In summary, this code listens for a button click. When the button is clicked, it sends a GET request to a specific URL using XMLHttpRequest. If the request is successful, the data is logged in both raw and parsed forms. If the request fails, the error is logged.

Convert the fetch code to xml http request :-

Lec 19 2:08:00

you can also ignore xml http request as it does not get used in industry now.

Doubt :- ① maine google par "coding blocks" search kiya, how it is working exactly? kya iss normal search mei bhi hum api use kar rahe hai jaise hamne ajax mei kiya?

② Ajax mei api data ko dom manipulate kar raha hai. kya ye data ko browser render karega? Like it used to do for html, css & js jik.